

DP及优化

zmwang



写在前面

- 这里默认大家掌握了使用朴素dp解决题目的方法
- 有些题目可能不会放题面，而直接放朴素转移方程



状态设计优化

容易被忽略的基础



HDU6327 Random Sequence

There is a positive integer sequence $a_1, a_2, \dots, a_n$ with some unknown positions, denoted by 0. Little Q will replace each 0 by a random integer within the range $[1, m]$ equiprobably. After that, he will calculate the value of this sequence using the following formula :

$$\prod_{i=1}^{n-3} v[\gcd(a_i, a_{i+1}, a_{i+2}, a_{i+3})]$$

Little Q is wondering what is the expected value of this sequence. Please write a program to calculate the expected value.

- $T \leq 10, n, m \leq 100, a_i \leq m$



HDU6327 Random Sequence

朴素想法是 $f_{i,a,b,c} * \gcd(a,b,c,d) \rightarrow f_{i+1,b,c,d}$, 就 n^5 了

但是我们只需要能根据状态知道 $\gcd(a,b,c,d)$ 就行啊?

把一维改成 $\gcd(a,b,c) \rightarrow$ 有一维需要 $\gcd(b,c)$ 来求 $\gcd(b,c,d) \rightarrow$ 有一维需要 c 来求 $\gcd(c,d)$

所以改成 $f_{i,\gcd(a,b,c),\gcd(b,c),c} * \gcd(a,b,c,d) \rightarrow f_{i+1,\gcd(b,c,d),\gcd(c,d),d}$

这样的状态有没有优化呢? 注意到 $\gcd(a,b,c) \mid \gcd(b,c) \mid c$, 而且三个都 ≤ 100

打表发现, 这样的三元组只有 1471 个, 因此复杂度 $1471nm$ 就能过了



[2021山东省队集训]button

【问题描述】

显示器上有 5 个数位，每个位置可以显示 0 到 9，也就是说显示器可以显示 0 到 99999 的这些数字。初始所有位置都是 0，系统从 $[L, R]$ 中随机选出一个数字 x ，现在我们要猜这个数字是几。

每一步，我们可以干两件事情中的一件：

1. 修改一个数位；
2. 问问当前数字和 x 的大小关系（返回大于、等于、小于）；

当我们确定 x 是几的时候，我们就 *win* 了。

现在我们要设计一个最优策略，请问我们最少需要多少步，才能猜到所有的 x ？

对于所有数据， $1 \leq test \leq 50, 1 \leq L < R \leq 99999$ 。



[2021山东省队集训]button

首先观察到操作次数很少（最多 45 次），要值域反转。

设 $l_{i,j}$ 表示目前区间右端点是 i 而且数字也是 i ， j 次最靠左能分开到哪里， $r_{i,j}$ 同理

考虑 $l_{i,j}$ 怎么求，需要枚举一个数字 k ，在 $r_{k,j-dis(i,k)-1} \geq i-1$ 的条件下找到最小的一个 $l_{k,j-dis(i,k)-1}$

我们发现 $dis(i,k) \leq 5$ ，因此实际上对于每个 i ， k 只有 2^5 类，因此所有的转移只有 $32n$ 类

每一类转移是将 i 中的一些位置改为任选，求这种数在某个第二维， r 的一个后缀的 min_l

所以再做一次压缩状态：对于一个一些位置任选，一些位置确定的五位数，可以将其压缩为 11^5 个状态之一，考虑维护每个压缩状态的 D_w 代表 min_l

因此考虑枚举 i 时对于 $m \leq 5$ ，将所有 $r_{k,j-m-1} = i-1$ 的加入压缩状态（每轮枚举这样的加入操作最多有 $5n$ 个）

这里将 m 做为 r 条件与 l 转移的相同第二维，即 $dis(i,k)$ ，因而对于每次加入操作，只需要 $\binom{5}{m}$ 的枚举 k 中 m 个数换成任选的状态 w ，更新这些压缩状态的 D_w 即可，这里最多一共是 $n * 2^5$ 个

时间复杂度为 $45 * (32n + 11^5 + 5n)$ ，求 r 同理



单调队列优化



经典题： 多重背包

$$i \in [0, n], j \in [0, m]$$

$$f_{i,j} = \max_{k=0}^{c_i} (f_{i-1,j-k*w_i} + k * v_i)$$

按照 $j \bmod w_i$ 分为 w_i 组

每组相当于 $g_{i,j} = \max_{k=0}^{c_i} (g_{i-1,j-k} + k * v_i)$, 其中 $g \in [0, \frac{m}{w_i}]$

令 $h_{i,j} = g_{i,j} - j * v_i$

则 $g_{i,j} = \max_{k=0}^{c_i} (h_{i-1,j-k} + j * v_i) = \max_{k=0}^{c_i} (h_{i-1,j-k}) + j * v_i$, 是个显然的滑动窗口, 因此每组复杂度 $O(\frac{m}{w_i})$, 总复杂度

$$O(n * \frac{m}{w_i} * w_i) = O(nm)$$



Luogu P5858 「SWTR-03」 Golden Sword

题目描述

制造一把金宝剑需要 n 种原料，编号为 1 到 n ，编号为 i 的原料的坚固值为 a_i 。

炼金是很讲究放入原料的顺序的，因此小 E 必须**按照 1 到 n 的顺序**依次将这些原料放入炼金锅。

但是，炼金锅的容量非常有限，它**最多只能容纳 w 个原料**。

所幸的是，**每放入一个原料之前**，小 E 可以从中取出一些原料，数量不能超过 s 个。

- 我们定义第 i 种原料的耐久度为：放入第 i 种原料时锅内的原料总数（包括正在放入的原料） $\times a_i$ ，则宝剑的耐久度为**所有原料**的耐久度之和。

小 E 当然想让他宝剑的耐久度尽可能得大，这样他就可以带着它进行更多的战斗，请求出耐久度的最大值。

对于 100% 的数据， $1 \leq s \leq w \leq n \leq 5 \times 10^3$ ， $|a_i| \leq 10^9$ 。对于 Subtask i 有 $|a_i| \leq 10^{i+1}$ 。



Luogu P5858 「SWTR-03」 Golden Sword

$f_{i,j}$ 代表考虑到第 i 个原料, 锅里有 j 个的最大值

$$f_{i,j} = \max(f_{i-1,k} + j * a_i)$$
$$(j - 1 \leq k \leq j + 1 - s)$$

把 $j * a_i$ 拿出来, 里面是个显然的滑动窗口

$$O(nw)$$



「SCOI2010」股票交易

最近 lxhgww 又迷上了投资股票，通过一段时间的观察和学习，他总结出了股票行情的一些规律。

通过一段时间的观察，lxhgww 预测到了未来 T 天内某只股票的走势，第 i 天的股票买入价为每股 AP_i ，第 i 天的股票卖出价为每股 BP_i （数据保证对于每个 i ，都有 $AP_i \geq BP_i$ ），但是每天不能无限制地交易，于是股票交易所规定第 i 天的一次买入至多只能购买 AS_i 股，一次卖出至多只能卖出 BS_i 股。

另外，股票交易所还制定了两个规定。为了避免大家疯狂交易，股票交易所规定在两次交易（某一天的买入或者卖出均算是一次交易）之间，至少要间隔 W 天，也就是说如果在第 i 天发生了交易，那么从第 $i+1$ 天到第 $i+W$ 天，均不能发生交易。同时，为了避免垄断，股票交易所还规定在任何时间，一个人的手里的股票数不能超过 MaxP 。

在第一天之前，lxhgww 手里有一大笔钱（可以认为钱的数目无限），但是没有任何股票，当然， T 天以后，lxhgww 想要赚到最多的钱，聪明的程序员们，你们能帮助他吗？

对于 100% 的数据， $0 \leq W < T \leq 2000, 1 \leq \text{MaxP} \leq 2000$

对于所有的数据， $1 \leq BP_i \leq AP_i \leq 1000, 1 \leq AS_i, BS_i \leq \text{MaxP}$



「SCOI2010」股票交易

$f_{i,j}$ 表示第 i 天持 j 股的最大收益

对于买股票: $f_{i,j} = \max_{(j-as_i) \leq k < j} (f_{i-w-1,k} - (j-k) * ap_i)$

发现对于每个 j , 要求一个 $\max_{(j-as_i) \leq k < j} (f_{i-w-1,k} + k * ap_i)$

是个滑动窗口

显然单调队列即可

卖股票同理



数据结构优化



CF1479B Painting the Array

- 给定一个长 n 数组 a ，对于所有将其每个元素染成黑色或白色的方案，求将白色与黑色元素子序列分别取出，相邻元素去重后的长度之和的最大/小值
- $n \leq 10^5$



CF1479B Painting the Array

以最大值为例，令 $f_{i,0/1,j}$ 表示考虑到第 i 个位置，颜色为 0/1，**另一种颜色**最近的数为 j 的最大值

$$f_{i,0,j} = f_{i-1,0,j} + [a_{i-1} \neq a_i] (j \neq a_{i-1})$$

$$f_{i,0,a_{i-1}} = \max(f_{i-1,0,a_{i-1}} + [a_{i-1} \neq a_i], f_{i-1,1,k} + [a_i \neq k])$$

因此开两棵线段树维护 $f_{i,0}$ 与 $f_{i,1}$ ，支持两个操作：

全局加 $[a_{i-1} \neq a_i]$

取另一边最大值加上 $[a_i \neq k]$ 后单点更新



[NOI2020] 命运

形式化的：给定一棵树 $T = (V, E)$ 和点对集合 $\mathcal{Q} \subseteq V \times V$ ，满足对于所有 $(u, v) \in \mathcal{Q}$ ，都有 $u \neq v$ ，并且 u 是 v 在树 T 上的祖先。其中 V 和 E 分别代表树 T 的结点集和边集。求有多少个不同的函数 $f: E \rightarrow \{0, 1\}$ （将每条边 $e \in E$ 的 $f(e)$ 值置为 0 或 1），满足对于任何 $(u, v) \in \mathcal{Q}$ ，都存在 u 到 v 路径上的一条边 e 使得 $f(e) = 1$ 。由于答案可能非常大，你只需要输出结果对 998,244,353（一个素数）取模的结果。

测试点约束

全部数据满足： $n \leq 5 \times 10^5$ ， $m \leq 5 \times 10^5$ 。输入构成一棵树，并且对于 $1 \leq i \leq m$ ， u_i 始终为 v_i 的祖先结点。



[NOI2020] 命运

考虑设计树型DP状态，在每个点处要记什么？

DP是从下往上合并，在做到每个点时，下面会有若干条没被满足的链伸出来

在任何时刻只要选出一条边，所有伸出来的链就都被解决了

所以伸出来的链要考虑最严格的那个条件——最深的上顶点深度

$f_{i,j}$ 代表考虑到 i ，最深的未满足链上顶点深度为 j 的方案数



[NOI2020] 命运

$f_{i,j}$ 代表考虑到 i ，最深的未满足链上顶点深度为 j 的方案数

考虑转移，如果加入那条链是最深的，转移是 $f_{i,j} \leftarrow \sum_{k=0}^{j-1} f_{i,k} * f_{v,j}$

否则是 $f_{i,j} \leftarrow \sum_{k=0}^j f_{v,k} * f_{i,j}$

如果对这条边染色，那么是 $f_{i,j} \leftarrow \sum_{k=0}^{dep_v-1} f_{v,k} * f_{i,j}$

对 f 做个前缀和得到 g ，那么上面合起来是

$$f_{i,j} \leftarrow g_{i,j-1} * f_{v,j} + g_{v,j} * f_{i,j} + g_{v,dep_v-1} * f_{i,j}$$



[NOI2020] 命运

这种按位相乘的形式，一般指向线段树合并

而恰好每个子树内实际上可能的不同状态只有子树内出现的链那么多个，所有点一共只有 m 个，满足线段树合并的套路

合并时在一边的树空的时候另一边需要区间乘一个前缀和，但是这没有关系，线段树合并的过程中维护一下就好了



斜率优化



「HNOI2008」玩具装箱

$$f_i = \min(f_j + (i - (j + 1) + \sum_{k=j+1}^i c_k - L)^2)$$

说明/提示

对于全部的测试点, $1 \leq n \leq 5 \times 10^4$, $1 \leq L \leq 10^7$, $1 \leq C_i \leq 10^7$ 。



「HNOI2008」玩具装箱

$$f_i = \min(f_j + ((i + s_i) - (j + 1 + s_j + L))^2)$$

设 $a_i = i + s_i$, $b_i = i + 1 + s_i + L$

$$f_i = \min(f_j + (a_i - b_j)^2)$$

$$f_i = \min(f_j + a_i^2 + b_j^2 - 2 * a_i * b_j)$$

对于转移点, $f_i - a_i^2 = f_j + b_j^2 - 2 * a_i * b_j$

注意到我们需要根据一堆 $(b_j, f_j + b_j^2)$ 求最小的 $f_i - a_i^2$

把 $f_j + b_j^2$ 设成 y_j , 把 $2 * a_i$ 设成 k_i , 就是 $y_j = (k_i * b_j + (f_i - a_i^2))$, 长得很像直线方程

这个式子就是斜率优化标准形式



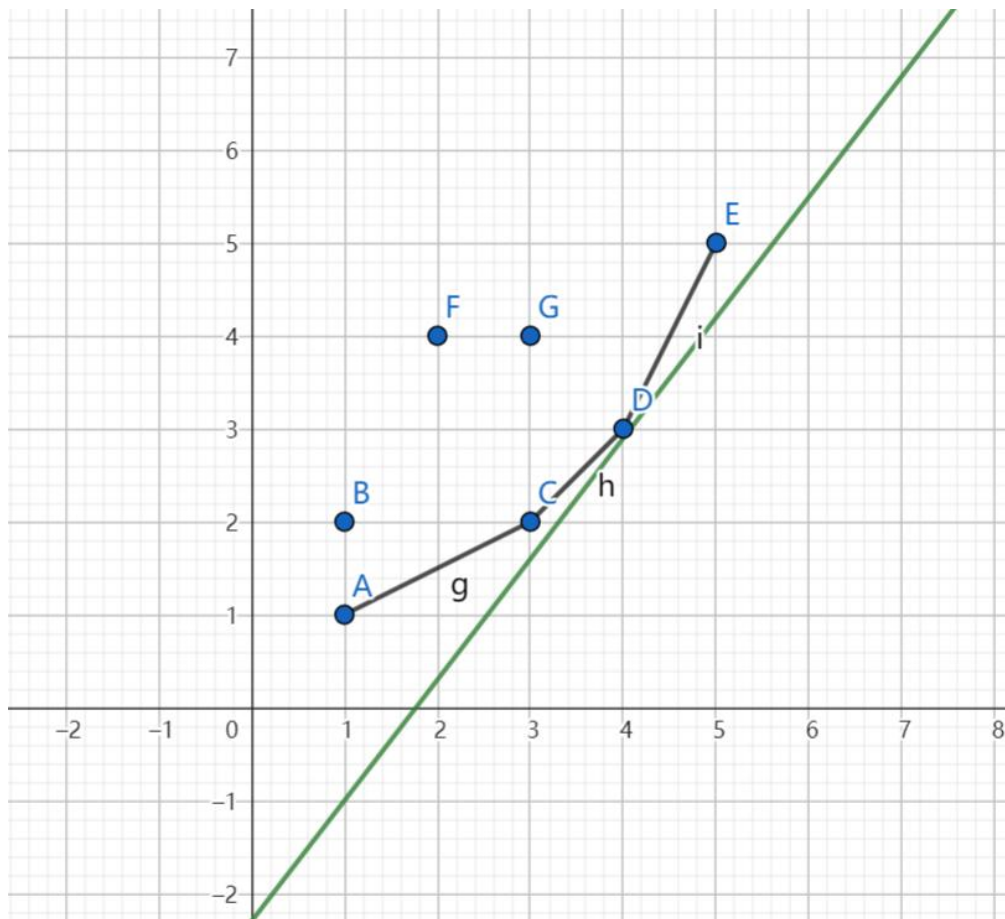
「HNOI2008」玩具装箱

要最小化的 $f_i - a_i^2$ 是什么？纵截距！

现在问题转化为：有一堆点 (b_j, y_j) ，找到过这些点做斜率是 k_i 的线，最大纵截距是多少



「HNOI2008」玩具装箱



「HNOI2008」玩具装箱

发现这个点一定在下凸壳上

现在回头看加点和斜率都是什么

设 $a_i = i + s_i$, $b_i = i + 1 + s_i + L$, 把 $2 * a_i$ 设成 k_i

k_i 与 b_i 均递增!



「HNOI2008」玩具装箱

- [Link](#)
- 一边插入一边维护下凸壳和答案点即可，复杂度 $O(n)$
- 对于一般的情况，可以使用二分+数据结构维护， $O(n\log n)$
- 一种更好写的方法：

回到 $f_i = \min(f_j + a_i^2 + b_j^2 - 2 * a_i * b_j) = a_i^2 + \min(f_j + b_j^2 - 2 * a_i * b_j)$

即求 $y = b_j * x + (f_j + b_j^2)$ 在 $x = -2 * a_i$ 的最小值

李超树即可。



[NOI2019] 回家路线

- [Link](#)



[NOI2019] 回家路线

- 把列车按出发时间排序
- 设 $f[i]$ 是最后坐第 i 辆车, $q[i]$ 时的最小烦躁值
- 则 $f[i] = \min(f[j] + \text{calc}(i, j))$
- 拆开 $\text{calc}(i, j)$, 斜率优化或者李超树均可



WQS二分/DP凸优化



经典题

- 有一个带权无向图，每条边是黑色或白色。求恰好有 k 条白边的最小生成树
- 至少需要一个 $O(m \log m \log w)$ 的做法



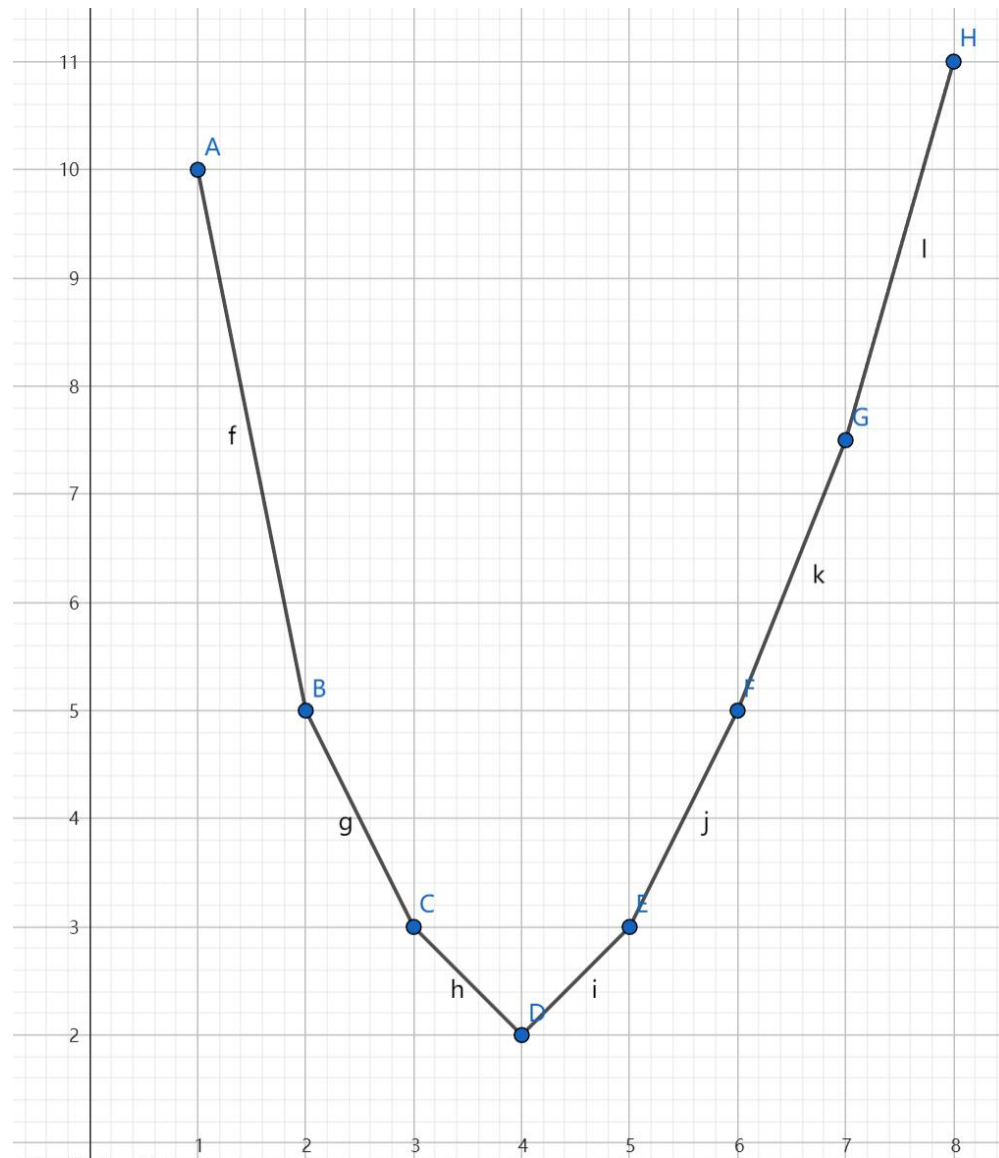
经典题

- 这甚至不是个DP?
- 对, WQS二分在优化DP的时候叫做DP凸优化, 但它不只能优化DP
- 我们能求什么?
- 没有限制的最小生成树, 没了。
- 我们还能发现什么?
- 容易感觉到白边太多或者太少都会让答案变大
- 打个表, 发现答案随着 k 是下凸的



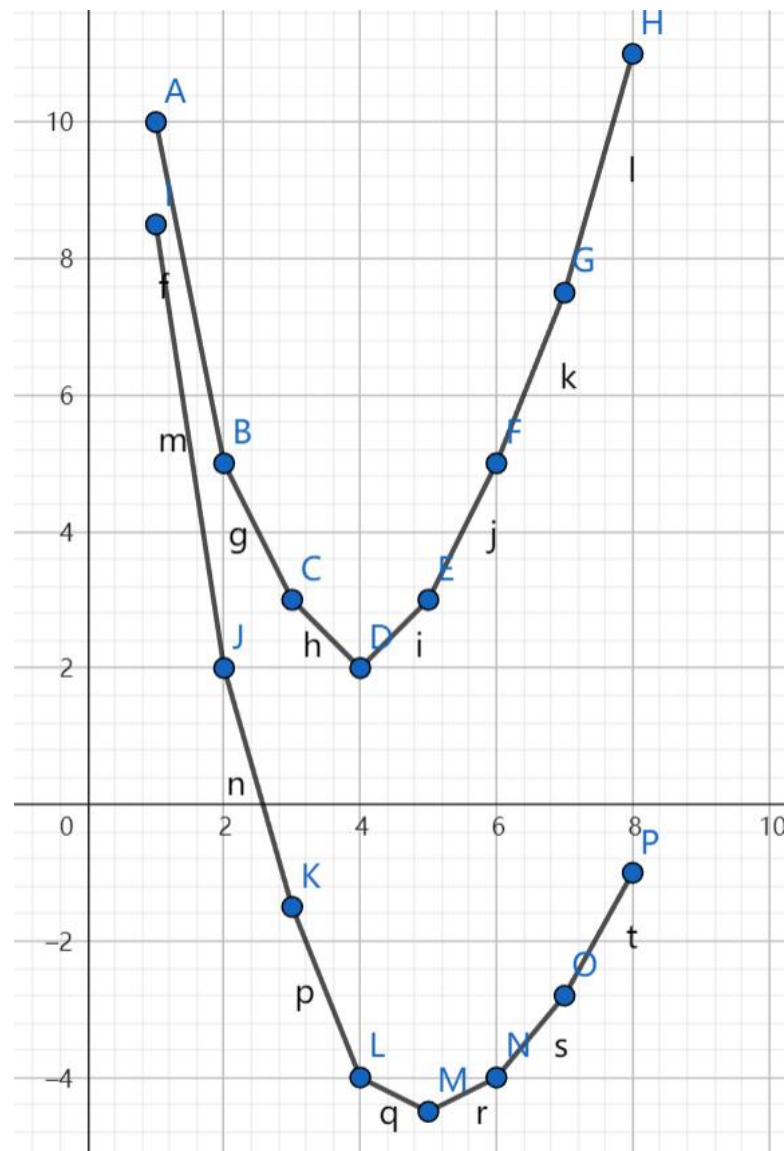
经典题

- 但是这样我们只能求出来最低的那个点，比如说是 $k=4$ 的那个点
- 如果我们要 $k=5$ 怎么办？
- 有什么办法能够让最低点变成5，而且保留下凸性，还能还原答案？
- 给每个白边添/减代价！



经典题

- 每个点下移 $2x$ 后的结果
- 相当于给每个白边减了2的代价
- 这样我们求出来的答案就是 $k=5$ 的时候了!
- 那怎么调整成我们想要的 k 呢
- 二分!
- 做完了。



实现细节

- 对于整数二分的时候，对于一个斜率，有可能有一整段都是极值点
- 在需要判无解等情况，需要以选的最多/选的最少各算一遍，确定这个限制确实在极值点区间里
- 比如最小生成树题里，Kruskal 算法时同边权优先选白边做一遍有 r 条白边，优先选黑边有 l 条，确认一下确实 $l < k < r$
- 在WQS二分的输出方案中，一般也需要用到这个以及后缀最大/小值，这个有兴趣者详见 CF125E MST Company



「九省联考 2018」林克卡特树

题目描述

小 L 最近沉迷于塞尔达传说：荒野之息（The Legend of Zelda: Breath of The Wild）无法自拔，他尤其喜欢游戏中的迷你挑战。

游戏中有一个叫做“LCT”的挑战，它的规则是这样子的：现在有一个 N 个点的树（Tree），每条边有一个整数边权 v_i ，若 $v_i \geq 0$ ，表示走这条边会获得 v_i 的收益；若 $v_i < 0$ ，则表示走这条边需要支付 $-v_i$ 的过路费。小 L 需要控制主角 Link 切掉（Cut）树上的**恰好** K 条边，然后再连接 K 条边权为 0 边，得到一棵新的树。接着，他会选择树上的两个点 p, q ，并沿着树上连接这两点的简单路径从 p 走到 q ，并为经过的每条边支付过路费 / 获取相应收益。

海拉鲁大陆之神 TemporaryDO 想考验一下 Link。他告诉 Link，如果 Link 能切掉合适的边、选择合适的路径从而使**总收益-总过路费**最大化的话，就把传说中的大师之剑送给他。

小 L 想得到大师之剑，于是他找到了你来帮忙，请你告诉他，Link 能得到的**总收益-总过路费**最大是多少。

对于全部的测试数据，保证有 $1 \leq N \leq 3 \times 10^5, 1 \leq x_i, y_i \leq N, |v_i| \leq 10^6$ 。

提示：题目并不难。



「九省联考 2018」林克卡特树

- 显然切完 k 条边后那 $k+1$ 个连通块的直径和就是答案
- 也就是转化为在树上选 $k+1$ 条点不交的路径，最大化权值和
- 这是一个树形 DP 实例，具体在此不表
- 这个函数是凸的，所以可以 WQS 二分（给每选一条链加一个代价，去掉一维）
- 「反正闲着也是闲着，我把 DP 数组打印出来吧！」
 - 你很快写了个暴力，把 $best[k]$ （表示选了 k 条链的最优解）打了出来
- 唔这个 DP 数组好难看啊，我来把它差分一下！
- 哎…我发现了什么了不得了的规律
 - 似乎差分之后的数组是递减的哎？」



决策单调性优化

基于Itst APIO2021课件



决策单调性

- 决策单调性，就是对于最优化DP，把取到那个点作为决策点，那么随着 i 增大， $dp[i]$ 的决策点是不降的
- 理论上应该用四边形不等式发现，实践中一般靠打表发现
- 有两类：
- 第一类，从上一层转移到这一层，比如说 $f[i][j]=\min(f[i-1][k]+g(j,k))$ ，随着 j 增大 k 不降
- 第二类，从本层前面转移到后面，比如说 $f[i]=\min(f[j]+g(i,j))$
- 我们把第一类叫做离线，第二类叫做在线



决策单调性

- 我们将问题标准化一下：
- 考虑一个 $n \times n$ 的矩阵，我们规定第 i 行第 j 列是从选择从 i 转移到 j 位置时的值
- 比如离线时 $a[i][j]$ 就是 $f[n-1][i] + \text{calc}(i, j)$
- 在线时 $a[i][j]$ 就是 $f[i] + \text{calc}(i, j)$
- 那么每一行的最小值点单调不降
- 对于离线，可以随意快速求矩阵任何一个值
- 对于在线，只有求出第 i 行的最小值即 $f[i]$ ，才能快速求 $a[i][*]$



决策单调性

- 有一个性质：如果只考虑前几个时，后面所有的决策也是单调的
- 反映到矩阵上：任一个子矩阵都有和主矩阵一样的单调性质
- 理论上这个也可以用四边形不等式推，实践中与前面一样，都是写完再看对不对



决策单调性

- 经典离线做法：分治
- 找中间一行，算出最小值点，两边分下去就行
- $O(n \log n)$
- 注：与原课件不同，这里我们不区分 n 和 m
- 分治做法还有一个额外的好处：解决某些calc难以计算的问题
- 例：CF868F



决策单调性

- 例：CF868F
- $\text{Calc}(i,j)$ 是 $i \sim j$ 的相等数对数量，没法快速直接求，但是支持快速插入删除，也即：可以在矩阵上快速四连通移动
- 则每一层分治时，由于决策点单调，所求点会单调的向下、向右移动，求出该层分治所有答案需要时间为 $O(n)$
- 因此可在 $O(kn \log n)$ 解决此题



决策单调性

- 经典在线做法：单调栈二分
- 因此我们维护每行目前的最小值点，形如112222334这样
- 用单调栈存储最小值点相同的区间
- 每次加入一个新行时一定更新的是一段后缀
- 因此弹栈到临界区间后，在临界区间内二分
- 每个区间入栈一次、出栈一次、入栈时有一个区间被二分
- $O(n \log n)$



决策单调性

- 有没有线性做法呢？SAMWK
- 有一个想法：如果我们能够知道所有偶数行的最小值点，只要扫一遍就知道所有奇数行的最小值点了（前提：这是个离线算法）
- 但是这样还是 $n\log n$ 啊
- 咋办呢？注意到一个事情，递归到下面的时候，列数没变，列数远远高于行数，扫一遍很浪费
- 引入reduce操作，在 $O(n+m)$ 时间内把 $n*m$ 矩阵重整为 $n*n$ 方阵



决策单调性

- 基本思想：删除不可能成为最小值点的列
- 算法过程：维护前 k 行中可能的列位置
- 首先令 $k=1$
- 如果 $a[k][k] > a[k][k+1]$ ，删除第 k 列， k 向上退一行
- 如果 $a[k][k] < a[k][k+1]$ ，要是 $k=n$ 就删除第 $k+1$ 列，否则 k 向下进一行
- k 最大是 n ， k 每次减一的代价都是删一列，时间复杂度 $O(n+m)$
- 正确性？



决策单调性

- $k > 1$ 时, 对每个 $1 \sim k-1$ 的 i , $a[i][i+1] > a[i][i]$
- 当 $a[k][k] > a[k][k+1]$ 时:
- 如果前面有一行最小值点是 k , 那么取前 k 列的子矩阵, 根据单调性可得第 $k-1$ 行最小值点也是 k , 与 $a[k-1][k] > a[k-1][k-1]$ 矛盾
- 删掉第 k 列后不一定有 $a[k-1][k] > a[k-1][k-1]$ 了, 因此 k 要向上退一行
- 当 $a[k][k] < a[k][k+1]$ 时, 自然可以往后进一行/进到底就删掉第 $k+1$ 列



决策单调性

- 能不能在线呢？
- Wilber&Eppstein
- 略过不表（我不会x）



决策单调性

- 在2D-1D中的应用
- w 的交叉优于包含或包含优于交叉 + w 的四边形不等式可以推出 f 的决策单调性
- 实践中，建议发现第一点后直接打表
- 推出 f 的决策单调性后的优化是trivial的



决策单调性

- 关于决策单调性优化的更多应用，有兴趣者可参阅彭斯进于 APIO2021 《决策单调性与四边形不等式》



「雅礼集训 2017 Day5」 珠宝

01背包， N 个物品，体积 C_i ，价值 V_i ，对容量 $1 \sim K$ 分别求解

数据范围与提示

对于 20% 的数据， $N, K \leq 10000$;

对于另外 20% 的数据， $C_i = V_i$;

对于 100% 的数据， $1 \leq N \leq 1000000, 1 \leq K \leq 50000, 1 \leq C_i \leq 300, 0 \leq V_i \leq 10^9$ 。



「雅礼集训 2017 Day5」 珠宝

唯一可能的突破口是 $C_i \leq 300$

注意到对于 C_i 相同的物品，显然 V_i 从大往小贪心选

因此对 C_i 相同的物品排序前缀和，则转移方程类似 $f_{j,i} = \max_{k=0}^{num_j} (f_{j-1,i-j*k} + sum_{j,k})$

仍然是按照 $i \bmod k$ 分 k 组，变成 $f_{j,i} = \max_{k=0}^{num_j} (f_{j-1,i-k} + sum_{j,k})$

随着 i 增大，决策点 k 是单调不升的。感性理解，在空间越来越有限的情况下，选择性价比靠后的元素就越来越不明智了。

这是离线问题，可以分治解决。每组时间复杂度 $O(\frac{K}{k} \log \frac{K}{k})$ ，总复杂度 $O(CK \log K)$



矩乘优化



[SDOI2015]序列统计

小C有一个集合 S ，里面的元素都是小于 m 的非负整数。他用程序编写了一个数列生成器，可以生成一个长度为 n 的数列，数列中的每个数都属于集合 S 。

小C用这个生成器生成了许多这样的数列。但是小C有一个问题需要你的帮助：给定整数 x ，求所有可以生成出的，且满足数列中所有数的乘积 $\bmod m$ 的值等于 x 的不同的数列的有多少个。

小C认为，两个数列 A 和 B 不同，当且仅当 $\exists i \text{ s.t. } A_i \neq B_i$ 。另外，小C认为这个问题的答案可能很大，因此他只需要你帮助他求出答案对 1004535809 取模的值就可以了。

输入格式

一行，四个整数， $n, m, x, |S|$ ，其中 $|S|$ 为集合 S 中元素个数。

第二行， $|S|$ 个整数，表示集合 S 中的所有元素。

对于 100% 的数据， $1 \leq n \leq 10^9$ ， $3 \leq m \leq 8000$ ， $1 \leq x < m$ 。

m 为质数，输入数据保证集合 S 中元素不重复。



[SDOI2015]序列统计

- SDOI怎么会这么良心？哦原来这是一轮Day1 (x
- 先求原根和离散对数把题目变成模意义背包
- 因为每个数都是同样在S内任选的，这就是S所对应多项式的n次幂
- 用ln&exp实现多项式快速幂即可



[NOI2020] 美食家

- 题面太长辣
- [Link](#)
- 注意 $w_i \leq 5$



[NOI2020] 美食家

- 首先注意到可以把点权放到入边里
- 如果边权都是1，没有美食节怎么做？
- 边权矩阵做 $(+, \max)$ 乘法就是走一步！
- 因此矩阵快速幂即可。
- 但是现在边权是5不是1怎么办？
- 拆边？会拆出来 $n+4m$ 个点，寄了
- 拆点！把每个点搞成五个点的链，最后一个点是真点就行



[NOI2020] 美食家

- 但是还有美食节怎么办?
- 注意到美食节只有200次，按段计算之后每次特殊处理?
- $O(k(5n)^3 \log T)$ ，寄了
- 经典trick：我们只需要每个时刻1号点到每个点的答案!
- 因此只需要维护一个 $1 \times 5n$ 的向量
- $O((5n)^3 \log T)$ 的处理出转移矩阵，每一段用目前答案向量乘过去，后半部分就是 $O(k(5n)^2 \log T)$ 了



完结撒花！

